LLM Inference & Serving: Quick-Revision Cheatsheet

1. Core Inference & Serving Bottlenecks

- **Prefill vs. Decode**: Prefill processes all input tokens in parallel. It is **compute-bound** (General Matrix-Matrix Multiply, `GEMM`), utilizing GPU Tensor Cores. Decode processes tokens sequentially one-by-one. It is **memory-bandwidth-bound** (General Matrix-Vector Multiply, `GEMV`), bottlenecked by HBM memory bus transfer speeds.
- **Arithmetic Intensity**: $I = \frac{\text{FLOPs}}{\text{Memory Traffic (Bytes)}}$. Bounded by Roofline Model: $\text{Performance} = \min(P_{\text{peak}}, I \cdot B_{\text{peak}})$.
- **Prefill**: $I \approx L_{\text{prompt}}$ (high weight reuse; compute-bound).
- **Decode**: $I \approx 1$ FLOP/Byte (GPU loads entire model weights for a single token computation; memory-bound).
- **SLA Latency Metrics**:
- $\text{TTFT} = \text{Queue Delay} + \text{Prefill Latency}$ (critical for user responsiveness).
- $\text{TPOT (ITL)} = \text{Decode Iteration Latency}$ (critical for reading comfort).
- $\text{End-to-End Latency} = \text{TTFT} + (L_{\text{output}} - 1) \times \text{TPOT}$.
- $\text{Throughput (TPS)} = \frac{\text{Total Output Tokens}}{\text{Execution Time}}$ (critical for offline batching).

2. KV Cache VRAM Footprint & Optimization

- **VRAM Sizing Formula**:

$$\text{VRAM}_{\text{KV}} = 2 \cdot b \cdot l \cdot s \cdot n_{\text{heads\_kv}} \cdot d_{\text{head}} \cdot \text{bytes\_per\_element}$$

- b: Batch size.
- l: Context length.
- s: Layers count.
- $n_{\text{heads\_kv}}$: KV Heads (1 for MQA, G for GQA, H for MHA).
- d_{head}: Head dimension (typically 128).
- bytes_per_element: 2 (FP16/BF16), 1 (FP8/INT8).
- **VRAM Reduction**: Grouped-Query Attention (GQA) reduces KV heads by grouping queries (e.g. 8:1 ratio), shrinking KV cache size by **8x** compared to Multi-Head Attention (MHA).
- **Virtual Memory Virtualization**:
- **PagedAttention**: Divides KV Cache into non-contiguous physical blocks (e.g. 16 tokens), mapping logical requests via a block table to eliminate internal and external VRAM fragmentation (wasted VRAM drops from $\approx 60\% - 80\%$ to $< 4\%$).
- **RadixAttention**: Caches KV states in a prefix Radix Tree. Reuses common prefixes (system prompts, static contexts) in multi-turn dialogues, dropping TTFT by $2\times - 5\times$.

3. Parallelism Scaling & Quantization Matrix

Multi-GPU Parallelism

- **Tensor Parallelism (TP)**: Splits layer weights (Column/Row linear layers) within a node. Lowest latency, but requires NVLink bandwidth due to All-Reduce syncs at every layer.
- **Pipeline Parallelism (PP)**: Splits layers sequentially across nodes. Scale-friendly, but introduces pipeline idle bubbles (mitigated by 1F1B micro-batching).
- **Context Parallelism (CP)**: Splits sequence length via ring-attention communication. Best for ultra-long context.

Quantization Trade-offs

Algorithm	Target	Pros	Cons	Production Choice
GPTQ	Weight-only (4-bit)	• Dynamic inverse Hessian updates; high accuracy at low bit-widths.	• Slow/complex calibration loop.	Offline model storage.
AWQ	Weight-only (4-bit)	• Protects 1% salient outlier channels dynamically.	• Needs dynamic de-quantization kernels in SRAM.	vLLM runtime default.
SmoothQuant	Weight + Activation (W8A8)	• Migrates activation scaling outliers to weights, enabling native INT8 GEMM.	• Slight perplexity drops on large models.	High-concurrency enterprise APIs.

4. Concluding Q&A & Interview Checklists

- **Speculative Verification Expected Speedup**:

$$E[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

- α: Average draft acceptance rate.
- γ: Draft lookahead length.
- **Troubleshooting Production Incidents**:
  1. **High TTFT / Normal TPOT**: High queue backlog or long input prefill processing. Fix: Enable Chunked Prefill and Prefix Caching.
  2. **Normal TTFT / High TPOT**: Memory bandwidth bottleneck, network serialization, or heavy Tensor Parallel All-Reduce comms. Fix: Quantize weights (AWQ) or scale TP nodes down (to avoid node-hop switches).
  3. **CUDA OOM on Servings Initialization**: Inefficient pre-allocated KV Cache page size. Fix: Adjust `gpu_memory_utilization` parameters in vLLM configs.

Legend:

- $N, L, |V|, d, b, s$: Parameter count, Context tokens, Vocabulary size, Model dimension, Batch size, Layers.